Neural Computing: The Basics

Topics

- Machine Learning: An Overview
- Neural Computing
- Neural Network Architecture
- Neural Network Application Development
- Learning Algorithms
- Programming Neural Networks
- Benefits and Limitations of Neural Networks
- Neural Networks and Expert Systems
- Summary
- Exercise

Note: Unit 7 is based on Chapter 17 of "Decision Support Systems and Intelligent Systems", E. Turban and J. Aronson, Prentice Hall

Machine Learning: An Overview

- *Machine learning:* methods that teach machines to support problem solving by applying historical cases
- ML is considered a part of AI. The immediate benefit is to enable AI programs to improve their performance over time
- Learning can improve the performance of AI products such as ES and robotics
- Learning in AI involves manipulation of symbols (not numeric information)
- No claims have been made about computers being able to learn as well as humans or in the same way
- The Artificial Neural Network (ANN) is the most commonly applied technology of the machine learning
- A major property of neural networks is their ability to learn from past experience and improve their performance levels

Machine Learning Methods

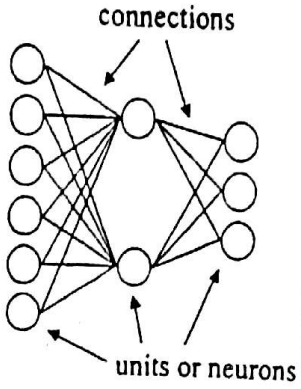
- Neural Computing
- Statistical methods
- Inductive Learning
- Case-based Reasoning and Analogical Reasoning
- Genetic Algorithms

An Overview of Neural Computing

- Neural Computing = Artificial Neural Networks (ANNs)
- Research on NC was pioneered by Frank Rosenblatt in the 1960s, who constructed a mechanism called a *perceptron*
- Renewed interest in the late 1980s because of advances in AI and progress in computer technology
- NC attempts to construct computers that mimic certain processing capabilities of the human brain
- It is an *intelligent* information processing technology
  - Knowledge is distributed throughout the network
  - Computations can be done in parallel (massive parallel processing)
  - Capability of learning
  - Ability to recognise patterns based on historical cases
  - Fast retrieval of large amounts of information

How Neural Networks Differ from Conventional Computers

- The **conventional computer** has two main components, a memory and some kind of processing device (that is a CPU).
- Information is represented in terms of structures of data and the way in which this information is processed depends on a program also stored in the computer's memory.
- The **neural network** is made up of a set of simple *processing units* (PE) connected together in a network. The units themselves have some similarity to biological neurons.
- Information is represented with *levels of activation* and the way in which it is processed all depends on the way in which *activation propagates* through the network, i.e. it all depends on the *connections* between the units.



The Biology Analogy

Biological Neural Networks

- The brain is composed of *Neurons* (Brain Cells)
- *Dendrites* provide inputs, *Axon* sends outputs
- *Synapses* increase or decrease connection strength and causes excitation or inhibition of subsequent neurons

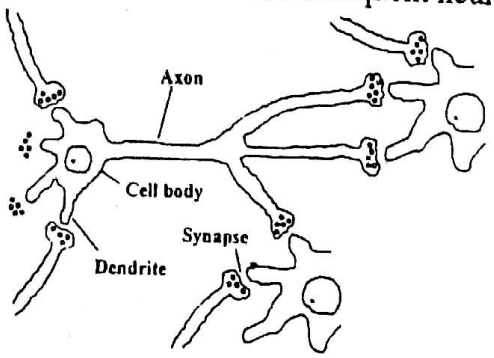


Fig 17.2. Simplified diagram of interconnected neurons

Biological Neuron

- The neuron produces regular, electrical impulses called *spikes*. These travel down the *axon* and across the *synapses*
- If the amount of *activation* (i.e. the number of spikes) which the neuron receives via the synaptic junctions at the end of its *dendrites* increases then the rate at which it sends out spikes (its *firing rate*) also increases, that is, it sends out more activation.
- If the amount of activation decreases, the firing rate decreases.
- Thus the firing rate reflects the amount of activation that the cell receives and its current *internal state*.
- To a crude approximation, the neuron behaves as an *activation-summing* device.

Artificial Neural Networks (ANN)

- ANN is a model that emulates a biological neural network
- Dual role of ANN
  - to improve the computer capabilities
  - to test various hypotheses about information processing in the brain
- Originally proposed as a model of the human brain's activities, but the human brain is *much more* complex than the model can capture
- ANN are hardware or software simulations of the parallel processes that involve processing elements (*artificial neurons*) interconnected in a network architecture
- Interconnected artificial neurons - Fig. 17.3

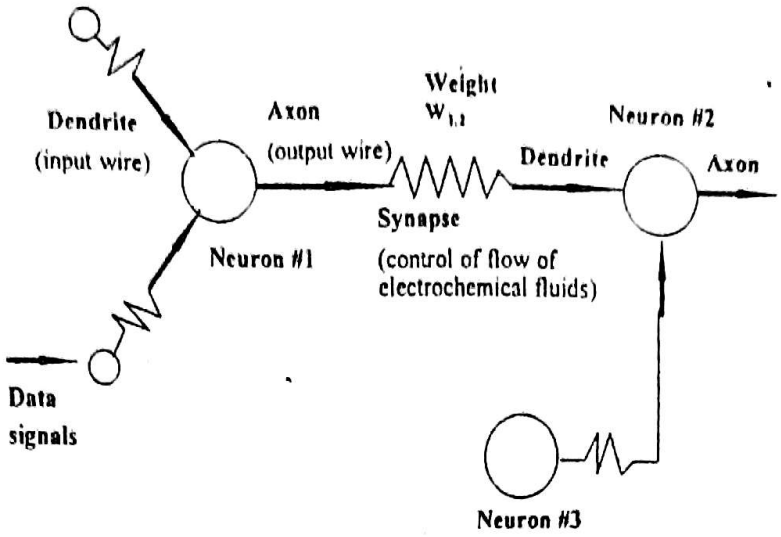


FIGURE 17.3 Three Interconnected Artificial Neurons

Artificial Neuron

- An (Artificial) Neural Network is composed of processing elements (artificial neurons), organised in some structure

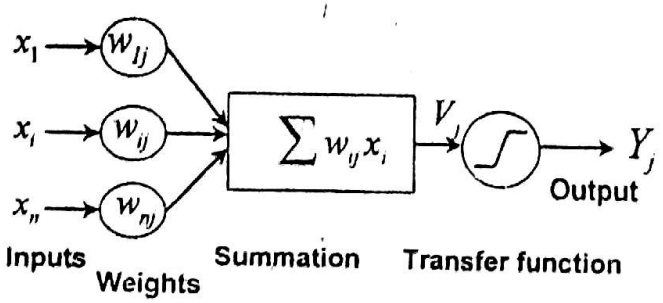


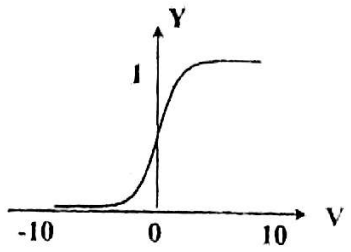
Fig. 17.4 Artificial Neuron

- *Weights* are key elements in an ANN. They express the relative importance of each input

Transfer (Activation) Functions

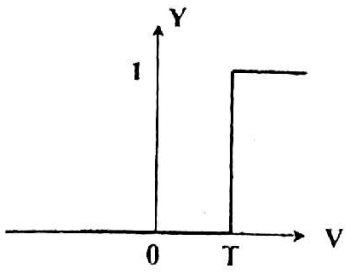
- Sigmoid Transfer Function

$$Y_j = \frac{1}{1 + e^{-V_j}}$$



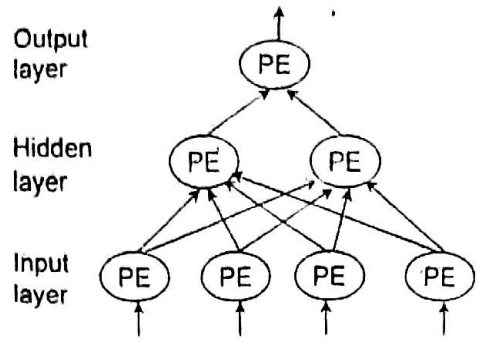
- Threshold Transfer Function

$$Y_j = \begin{cases} 1, & \text{if } V_j > T_j \\ 0, & \text{else} \end{cases}$$

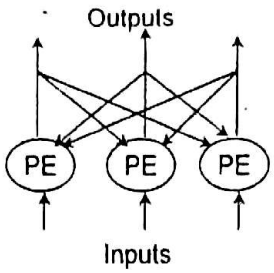


Neural Network Architectures

- *Associative memory* - ability to recall complete situations from partial information
- ANN are associative memory systems that *correlate* input data with stored information
- **Multilayer Architectures**
  - *Input Layer:* input processing elements (PE) only distribute information
  - *Hidden Layer(s):* are not observable
  - *Output Layer:* processing elements relay the results out of the net
- **Feedforward Architectures:** neuron outputs feed forward to subsequent layers
- **Recurrent Architectures**: neuron outputs feed back as neuron inputs



- Fig 17.5. **Multilayer Feedforward Network**



CT34B/Unit **Fig 17.10 Single Layer Recurrent Network** 13

ANN Learning

- Learning for ANN involves *adjusting* the weights
- The learning tasks are:

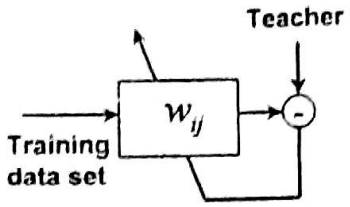
1. Compute Outputs
2. Compare Outputs with Desired Targets
3. Adjust Weights and Repeat the Process

- Learning starts by setting the initial weights by either some rules or randomly
- Delta = *error* between the actual output Y and desired one Z
- Objective is to *Minimise* the Delta
- The learning process changes the weights to reduce the Delta
- The weights change in response to *training* data
- Different learning algorithms

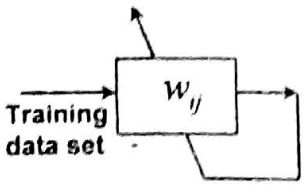
CT34SYUD 14

Learning Algorithms

- Two major categories based on *Input Format*
  - Binary-valued (0s and 1s)
  - Continuous-valued
- Two basic learning strategies
  - Supervised Learning
  - Unsupervised Learning



Supervised Learning



Unsupervised Learning

- **Supervised Learning**
  - Assumes the availability of a teacher who classifies the training data into classes
  - For a set of inputs with known (desired) outputs
  - Delta is used to calculate corrections to the weights
  - Examples:
    - Delta rule, Backpropagation
- **Unsupervised Learning**
  - Only inputs are shown to the network
  - Network is self-organising, heuristic algorithms
  - Number of categories into which the network classifies the inputs can be controlled by varying certain parameters in the network
  - Examples:
    - Principal component analysis, Competitive learning

Example of Supervised Learning Using Delta Rule

- Single neuron with two inputs X_1 and X_2, and output Y, learning the inclusive OR operation
- Truth table: Z is the Desired Output

Case	X_1	X_2	Z
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	1

- A threshold activation function evaluates the summation of input values
- Threshold value T=0.5: transfer function sets Y to 1 if the weighted sum of inputs is greater than 0.5; otherwise Y=0
- Weights are initialised randomly

- Iterative process of updating the weights W1 and W2 uses a *delta rule*. It updates weights by an amount proportional to the difference between the actual and the desired output

  Delta = Z - Y
- The updated weights at each iterations are

  W_i(final) = W_i(initial) + Alpha × Delta × X_i

  where Alpha is a *learning rate* parameter
- If the delta (error) is very small, then the weight changes are also very small. In the limit the error is zero and the weights are not changed at all
- The *learning rate* parameter is set low (Alpha=0:2)
- With a sufficiently small learning rate, a good result could be achieved
- If too high learning rate is used, the learning procedure will diverge, producing ever greater errors
- Results of learning are shown on Table 17.1

TABLE 17.1 Example of Supervised Learning

Step	X_1	X_2	Z	Initial W_1	Initial W_2	Y	Delta	Final W_1	Final W_2
1	0	0	0	0.1	0.3	0	0.0	0.1	0.3
	0	1	1	0.1	0.3	0	1.0	0.1	0.5
	1	0	1	0.1	0.5	0	1.0	0.3	0.5
	1	1	1	0.3	0.5	1	0.0	0.3	0.5
2	0	0	0	0.3	0.5	0	0.0	0.3	0.5
	0	1	1	0.3	0.5	0	1.0	0.3	0.7
	1	0	1	0.3	0.7	0	1.0	0.5	0.7
	1	1	1	0.5	0.7	1	0.0	0.5	0.7
3	0	0	0	0.5	0.7	0	0.0	0.5	0.7
	0	1	1	0.5	0.7	1	0.0	0.5	0.7
	1	0	1	0.5	0.7	0	1.0	0.7	0.7
	1	1	1	0.7	0.7	1	0.0	0.7	0.7
4	0	0	0	0.7	0.7	0	0.0	0.7	0.7
	0	1	1	0.7	0.7	1	0.0	0.7	0.7
	1	0	1	0.7	0.7	1	0.0	0.7	0.7
	1	1	1	0.7	0.7	1	0.0	0.7	0.7

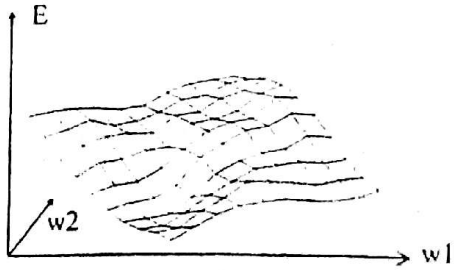
Parameters: alpha = 0.2; threshold = 0.5.

Back Propagation

- *Back propagation* is the most widely used learning for feed-forward multilayer nets with one or more hidden layers
  - The basic idea of backpropagation is that, we can change the weights by computing the error of the output units and propagating this error *backwards* through the network
  - The sum-squared error for the output layer E measures how well the current behaviour approximates the desired one

$$E = \frac{1}{2}\sum_{output}(Z_j - Y_j)^2$$

  - The *error surface* represents the error for all possible weights



- The aim of learning is to reduce the error to an absolute minimum. This means moving downhill on the error surface until the lowest point is reached.
- A well-known search strategy is the *gradient descent*.
- **Back propagation learning algorithm** is a *gradient descent technique* with *backward error propagation*

1. Start at the output layer and work backward to the hidden layers recursively. Adjust the weights by

$$W_{ij}(t+1) = W_{ij}(t) + \Delta W_{ij}$$

where $W_{ij}(t)$ is the weight from i-th to j-th PE at time t

ΔW_{ij} is the weight adjustment

2. The weight adjustment is computed by

$$\Delta W_{ij} = \alpha \delta_j Y_i$$

where α is the learning rate $0 < \alpha < 1$

δ_j is the error gradient at the j-th PE

3. The error gradient for the output PE (sigmoid function) is

$$\delta_j = Y_j(1-Y_j)(Z_j - Y_j)$$

4. The error gradient for the hidden PE (sigmoid function) is

$$\delta_j = Y_j(1-Y_j)\sum_k \delta_k W_{kj}$$

5. Repeat iterations until convergence in terms of selected error criterion. An iteration includes presenting an instance, calculating the outputs and modifying weights.

Features of Back-propagation

- Back-propagation is a simple procedure, easy to use, and it can be used to learn to approximate any input/output map
- The key disadvantage is that learning is slow and require lots of training data (typically three times more training samples than network weights)
- *Bias:* to improve learning by adding an additional level of activation independently of the inputs

Neural Network Applications Development

- The development process of ANN is similar to the design of traditional Computer based IS, but some steps are unique
- Main steps of ANN Application Development Process
  1. Collect Data
  2. Separate into Training and Test Sets
  3. Define a Network Structure
  4. Select a Learning Algorithm
  5. Set Parameters, Values, Initialise Weights
  6. Transform Data to Network Inputs
  7. Start Training, Determine and Revise Weights
  8. Stop and Test
  9. Implementation: Use the Network with New Cases

Programming Neural Networks

- An ANN can be programmed with
  - A programming language
  - A programming tool
- Tools (shells) incorporate
  - NN architectures
  - Training algorithms
  - Transfer functions
- May still need to
  - Program the layout of the database
  - Partition the data (test data, training data)
  - Transfer the data to files suitable for input to an ANN tool

Neural Network Applications Development

- The development process of ANN is similar to the design of traditional Computer based IS, but some steps are unique
- Main steps of ANN Application Development Process
  1. Collect Data
  2. Separate into Training and Test Sets
  3. Define a Network Structure
  4. Select a Learning Algorithm
  5. Set Parameters, Values, Initialise Weights
  6. Transform Data to Network Inputs
  7. Start Training, Determine and Revise Weights
  8. Stop and Test
  9. Implementation: Use the Network with New Cases

Programming Neural Networks

- An ANN can be programmed with
  - A programming language
  - A programming tool
- Tools (shells) incorporate
  - NN architectures
  - Training algorithms
  - Transfer functions
- May still need to
  - Program the layout of the database
  - Partition the data (test data, training data)
  - Transfer the data to files suitable for input to an ANN tool

Neural Network Hardware

- Advantages of massive parallel processing
- Greatly enhances performance
- Possible Hardware Systems for ANN Training
  - Faster general purpose computers
  - General purpose parallel processors
  - Neural chips
  - Acceleration boards

Benefits of Neural Networks

- Usefulness for pattern recognition, learning, classification, generalisation and abstraction, and the interpretation of incomplete and noisy inputs
- Specifically - character, speech and visual recognition
- Potential to provide some of human problem solving characteristics
- Ability to tackle new kinds of problems
- Robustness
- Fast processing speed
- Flexibility and ease of maintenance
- Powerful hybrid systems
- Suitable for complex decision support

Limitations of Neural Networks

- Do not do well at tasks that are not done well by people, as mathematical and data processing tasks
- Lack explanation capabilities, no justification for the results
- Limitations and expense of parallel hardware technology restrict most applications to software simulations
- Training times can be excessive and tedious
- Usually requires large amounts of training and test data

Neural Networks and Expert Systems

- ANN and ES technologies are so different that they can *complement* each other
- Expert systems represent a logical, symbolic approach
- Neural networks are model-based and use numeric and associative processing
- Main features of each (Table 17.2)

TABLE 17.2 Comparing Expert Systems and Artificial Neural Networks.

	Expert Systems	Artificial Neural Networks
1	Have user development facilities, but systems should preferably be developed by skilled developers because of knowledge acquisition complexities.	Have user development facilities and can be easily developed by users with a little training.
2	Takes a longer time to develop. Experts must be available and willing to articulate the problem-solving process.	Can be developed in a short time. Experts only need to identify the inputs, outputs and a wide range of samples.
3	Rules must be clearly identified. Difficult to develop for intuitively-made decisions.	Does not need rules to be identified. Well suited for decisions made intuitively.
4	Weak in pattern recognition and data analysis such as forecasting.	Well suited for such applications but need wide range of sample data.
5	Not fault tolerant.	Highly fault tolerant.
6	Changes in problem environment warrant maintenance.	Highly adaptable to changing problem environment.
7	The application must fit into one of the knowledge representation schemes (explicit form of knowledge).	ANNs can be tried if the application does not fit into one of ES's representation schemes.
8	The performance of the human expert who helped create the ES places a theoretical performance limit for the ES.	ANNs may outperform human experts in certain applications such as forecasting.
9	Have explanation systems to explain why and how the decision was reached. Required when the decision needs explanation to inspire confidence in users. Recommended when the problem-solving process is clearly known.	Have no explanation system and act like black boxes.
10	Useful when a series of decisions is made in the form of a decision tree and when, in such cases, user interaction is required.	Useful for one-shot decisions.
11	Useful when high-level human functions such as reasoning and deduction need to be emulated.	Useful when low-level human functions such as pattern recognition need to be emulated.
12	Are not useful to validate the correctness of ANN system development.	In certain cases are useful to validate the correctness of ES development.
13	Use a symbolic approach (people-oriented).	Use a numeric approach (data-oriented).
14	Use logical (deductive) reasoning.	Use associative (inductive) reasoning.
15	Use sequential processing.	Use parallel processing.
16	Are closed.	Are self-organizing.
17	Are driven by knowledge.	Are driven by data.
18	Learning takes place outside the system.	Learning takes place within the system.
19	Are built through knowledge extraction.	Are built through training, using examples.

Source: Items 1-12 are from J. R. Slater et al. "On Selecting Appropriate Technology for Knowledge Systems," *Journal of Systems Management*, October 1993, p. 15.

Summary

- ANN are conceptually similar to biological systems
- Human brain is composed of billions of neuron cells, grouped in interconnected clusters
- Neural systems are composed of interconnected processing elements called artificial neurons that form an artificial neural network
- Can organise an ANN many different ways
- Weights express the relative strength (or importance) given to input data
- Each neuron has an activation value
- An activation value is translated to an output through a transformation (transfer) function
- Artificial neural networks learn from historical cases

- Learning is done with algorithms
- Supervised and unsupervised learning strategies are used
- Testing is done by using historical data
- Neural computing is frequently integrated with traditional
- Many ANNs are built with tools that include the learning algorithm(s) and other computational procedures
- Adapted themselves to parallel processing. However, most run on standard computers
- Parallel processors can improve the training and running of neural networks
- Neural computing excels in pattern recognition, learning, classification, generalisation and abstraction, and interpretation of incomplete input data

- ANN do well at tasks done well by people and *not* so well at tasks done well by traditional computer systems
- Machine learning describes techniques that enable computers to learn from experience
- Machine learning is used in knowledge acquisition and in inferencing and problem solving
- Machines learn differently from people; usually not as well
- Neural networks are useful in data mining

Exercise

- 1. Study the neural network development shell HavBpETT (from PC Lab menu). Read the Tutorial section in the HELP file for creating, training and consulting (testing) a network.
- 2. Run the XOR example with the default parameters, training and consulting files and analyse the performance of the net (xorbest.net)
- 3. Change the number of hidden neurons, the type of the activation functions and analyse the differences in the learning process and behaviour of the changed network
- 4. Build your feedforward NN that solves the inclusive OR problem (use the training data from this lecture)
- 5. Write a short report describing your OR net, its learning and its performance. Provide the consultation report generated by the shell to illustrate your report.